



CEBU INSTITUTE OF TECHNOLOGY
U N I V E R S I T Y

IT342-G4

Systems Integration and Architecture 1

Software Requirements Specification (SRS)

Project Title: BarberConnect

Prepared By: Cabasag, Ma. Melessa Villas

Version: 2.0.0

Date: 31/05/2026

Status: Updated Draft

Revision History Table

Version	Date	Author	Changes Made	Status
1.0.0	03/02/2026	Cabasag, Ma. Melessa Villas	Initial FRS/SRS draft focused on mini app registration and authentication.	Draft
2.0.0	31/05/2026	Cabasag, Ma. Melessa Villas	Updated scope to BarberConnect full-stack platform; added customer, barber, and admin requirements; updated stack, system models, data model, and acceptance criteria to match current repository and SDD.	Updated Draft

Table of Contents

1. Introduction	2.5 Design and Implementation Constraints
1.1 Purpose	2.6 Assumptions and Dependencies
1.2 Scope	3. External Interface Requirements
1.3 Definitions, Acronyms, and Abbreviations	4. System Features and Functional Requirements
1.4 References	5. Non-Functional Requirements
2. Overall Description	6. Data Requirements
2.1 Product Perspective	7. Acceptance Criteria
2.2 Product Functions	8. System Models
2.3 User Classes and Characteristics	9. Requirements Traceability Matrix
2.4 Operating Environment	10. Appendices

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional, non-functional, interface, data, and system model requirements for BarberConnect. BarberConnect is a web and Android-based barbershop management platform that connects customers, barbers, and administrators in a single system for booking, catalog management, social posting, leave monitoring, and income tracking.

This updated SRS replaces the previous mini app authentication-only scope and aligns the requirements with the current BarberConnect implementation, updated SDD, and repository structure.

1.2 Scope

BarberConnect supports the complete grooming service workflow from account access to appointment completion. The system includes customer booking, barber schedule and catalog management, admin monitoring, social feed interaction, feedback, and automated income recording after completed services.

Included Features	Description
Authentication and Accounts	Register, login, Google sign-in where supported, JWT-secured access, and profile management.
Role-Based Access Control	Customer, Barber, and Admin access rules for both API and UI screens.
Customer Booking	Browse barbers and haircut styles, select date/time, choose payment method, and view appointment status.
Barber Management	Manage profile, haircut catalog, availability, appointments, posts, feedback, leave, and income dashboard.
Admin Monitoring	Manage barber accounts, approve or decline leave requests, set price policies, and monitor platform statistics/revenue.
Social Feed	Barber posts with image upload; customers can view, comment, and react.
Income Tracking	After appointment completion, record 80% barber share and 20% owner/platform share.

Excluded / Out of Scope	Reason
Real-time bank or card gateway integration	Only cash and GCash/e-wallet style payment selection are required for the academic implementation.
SMS gateway integration	Notifications may be future work; core app flow does not depend on SMS.
AI haircut recommendation	Not required for the current BarberConnect scope.
Multi-branch enterprise management	The design may support future extension but only the current barbershop scope is required.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
API	Application Programming Interface used by web and mobile clients to communicate with the backend.
JWT	JSON Web Token used to authorize protected requests.
RBAC	Role-Based Access Control; permissions depend on CUSTOMER, BARBER, or ADMIN role.
Firestore	Firebase Cloud Firestore NoSQL database used for application documents.
Cloudinary	Image storage service used for uploaded post/profile/catalog images.
OAuth2	Authentication protocol used for Google sign-in support.
GCash	E-wallet payment method selection used by customers for bookings.
Appointment Status	Booking state such as pending, confirmed, completed, cancelled, or no-show.

1.4 References

- BarberConnect GitHub repository README: project overview, technology stack, architecture, setup, and key features.
- BarberConnect_ERD.md: Firestore/NoSQL entity list and relationships.
- Updated System Design Document (SDD) v4.0.0 for BarberConnect.
- Original uploaded SRS/FRS document, version 1.0, used as the base document to replace and expand.

2. Overall Description

2.1 Product Perspective

BarberConnect is a multi-platform system composed of a React/Vite web frontend, Android Kotlin mobile application, Spring Boot REST backend, Firebase Authentication, Cloud Firestore database, and Cloudinary image storage. The backend follows a vertical slice structure where each feature owns its controller/service/model logic, such as auth, appointment, barber, catalog, admin, income, social, and shared services.

2.2 Product Functions

1. Allow users to register and log in securely through email/password and supported Google sign-in flows.
2. Route users to role-specific dashboards after authentication.
3. Allow customers to browse barber profiles, social feed posts, haircut services, ratings, and availability.
4. Allow customers to create bookings by selecting barber, haircut style, style options, date/time, and payment method.
5. Prevent duplicate or invalid bookings through backend validation and Firestore transaction rules.
6. Allow barbers to manage profile information, GCash number, availability, leave requests, appointment statuses, posts, and haircut catalogs.
7. Allow barbers to view appointment lists, income summaries, and feedback information.
8. Allow admins to manage barber accounts, price policy, leave approvals, revenue, appointment monitoring, and system statistics.
9. Allow creation of social feed posts with images and support comments/reactions.
10. Create income records after completed appointments using the required 80/20 split.

2.3 User Classes and CharacteristicsW

User Class	Main Responsibilities	Technical Skill Level
Customer	Register/login, browse posts and barbers, book appointments, choose payment method, track bookings, submit feedback, manage profile.	Basic smartphone or web browsing skills.
Barber	Manage catalog, schedule, leave requests, appointments, profile, posts, replies, feedback, and income dashboard.	Basic to moderate system usage skills.
Admin	Create/manage barber accounts, set price policies, approve/decline leave requests, monitor revenue/statistics, and supervise platform activity.	Moderate administrative skill level.

2.4 Operating Environment

Layer	Operating Environment / Tools
Backend	Java 17, Spring Boot 3.5.x, Maven, Firebase Admin SDK, Cloudinary SDK, JWT libraries.
Database	Cloud Firestore in Firebase native mode.

Authentication	Firebase Authentication plus Google OAuth2 and custom JWT handling.
Web Frontend	React 18/19, TypeScript, Vite, Axios, React Router, Tailwind CSS, Firebase JS SDK.
Mobile App	Android Kotlin, MVVM, Retrofit2, OkHttp3, DataStore, Firebase Android SDK, Glide.
Deployment	Backend deployed to Railway or similar Java hosting; web deployed through Vercel/Netlify; Android distributed as APK.
Devices	Desktop browser, mobile browser, and Android devices API 26+ for current mobile implementation.

2.5 Design and Implementation Constraints

- All protected API operations must verify the user token and role before processing requests.
- The system must use Cloud Firestore document collections rather than relational SQL tables for the current implementation.
- Image uploads must be handled through Cloudinary, and stored records must keep only safe media URLs and metadata.
- The booking module must validate barber leave and appointment conflicts before confirming a schedule.
- Income records must be generated only for completed appointments, not for pending or cancelled bookings.
- No secret keys, Firebase service accounts, Cloudinary secrets, or production tokens must be committed to the repository.
- Android real device testing must use the correct backend base URL for the local network or deployed server.

2.6 Assumptions and Dependencies

- Users have internet access when using web or mobile features.
- Firebase, Firestore, and Cloudinary services are available during system operation.
- The backend server is running and reachable by both web and Android clients.
- Admin account creation or bootstrap is configured before demonstration or deployment.
- Barbers maintain accurate availability and leave request information.
- Customers provide valid booking details and payment method selection.

3. External Interface Requirements

3.1 User Interface Requirements

Interface	Requirements
Customer Web/Mobile	Login/register, dashboard feed, available barbers, booking page, my bookings, profile page, feedback form, logout.
Barber Web/Mobile	Dashboard, schedule/calendar, appointment list, catalog management, post creation modal, profile, income, leave request, logout.
Admin Web Dashboard	Statistics, barber account management, leave request approval, schedule monitoring, price setting, revenue monitoring, logout.
Responsive Design	Screens must support mobile width, tablet layout, and desktop dashboard layout with readable forms and buttons.

3.2 Software Interface Requirements

External Software / Service	Interface Requirement
Firebase Authentication	Authenticate users and map identity to backend user profile and role.
Cloud Firestore	Store users, barber profiles, appointments, catalog, posts, comments, reactions, feedback, leave requests, income, and admin settings.

Cloudinary	Upload and retrieve image URLs for posts, profile images, and haircut catalog images.
Spring Boot REST API	Expose JSON endpoints under /api/v1 and apply security validation.
Android Retrofit Client	Consume backend API endpoints using correct base URL and Bearer token.
React Axios Client	Consume backend API endpoints and store/manage auth token client-side.

3.3 Communication Interface Requirements

- All production requests must use HTTPS/TLS.
- API requests and responses must use JSON format.
- Protected requests must send Authorization: Bearer <token>.
- Error responses must include a clear message that the client can show to the user.
- Network errors must be handled gracefully on both web and mobile clients.

4. System Features and Functional Requirements

4.1 Feature 1: Authentication and Account Management

The system shall allow secure account creation, login, logout, token validation, and profile access for customers, barbers, and admins.

- FR-1.1 The system shall allow a new customer to register using first name, last name, email, phone number, and password.
- FR-1.2 The system shall allow an admin to create or manage barber accounts when required by the workflow.
- FR-1.3 The system shall authenticate users using email/password and supported Google sign-in where configured.
- FR-1.4 The system shall return the authenticated user role so the client can redirect to the correct dashboard.
- FR-1.5 The system shall reject invalid credentials, duplicate email registration, missing fields, and weak password inputs.
- FR-1.6 The system shall provide a /me-style profile check for active sessions.

4.2 Feature 2: Role-Based Access Control

The system shall enforce different permissions for CUSTOMER, BARBER, and ADMIN users.

- FR-2.1 Customers shall access customer dashboards, bookings, feedback, profile, and social feed actions.
- FR-2.2 Barbers shall access barber dashboard, appointments assigned to them, catalog, posts, leave requests, feedback, and income views.
- FR-2.3 Admins shall access admin-only account management, leave approval, price settings, statistics, and revenue monitoring.
- FR-2.4 The backend shall reject requests where the user role does not match the required endpoint permission.

4.3 Feature 3: Customer Barber Discovery and Social Feed

Customers shall be able to discover barbers through profiles, posts, ratings, and service listings.

- FR-3.1 The system shall display available barbers with profile details, ratings, and service catalog information.
- FR-3.2 The system shall display barber-created posts with captions and images.
- FR-3.3 Customers shall be able to comment and react to posts when logged in.
- FR-3.4 The system shall show readable names rather than raw IDs where customer or barber names are available.

4.4 Feature 4: Appointment Booking

Customers shall create appointments by choosing a barber, haircut style, optional add-ons, date/time, and payment method.

- FR-4.1 The system shall allow customers to select one barber and one haircut style for a booking.
- FR-4.2 The system shall allow customers to select style options/add-ons when available.
- FR-4.3 The system shall compute total price from the base price and selected options.
- FR-4.4 The system shall validate that the selected date/time is not already booked and is not within an approved leave period.
- FR-4.5 The system shall store appointment records with customer, barber, haircut style, date/time, total price, payment method, and status.
- FR-4.6 The customer shall be able to view booking status under My Bookings.

4.5 Feature 5: Barber Appointment and Schedule Management

Barbers shall manage appointment records and schedule availability.

- FR-5.1 The system shall show upcoming and past appointments for the logged-in barber.
- FR-5.2 The barber shall be able to update appointment status according to allowed workflow states.
- FR-5.3 The barber shall be able to submit leave requests with date and reason.
- FR-5.4 Approved leave days shall block customer booking for the affected barber and date/time range.
- FR-5.5 The system shall display Free, Booked, or Leave schedule indicators where applicable.

4.6 Feature 6: Haircut Catalog Management

Barbers shall manage haircut styles and optional service add-ons.

- FR-6.1 Barbers shall be able to create, update, and deactivate haircut styles.
- FR-6.2 Each haircut style shall include name, description, base price, duration, active status, and optional image URL.
- FR-6.3 Barbers shall be able to create and update style options/add-ons with additional price and time.
- FR-6.4 The system shall enforce admin-defined minimum and maximum price policies when configured.

4.7 Feature 7: Payment Selection and Income Tracking

The system shall record payment method selection and income share after service completion.

- FR-7.1 Customers shall select Cash or GCash/e-wallet style method during booking.
- FR-7.2 The system shall not create an income record until the appointment is marked completed.
- FR-7.3 Upon completion, the system shall compute barber share as 80% of total price and owner/platform share as 20%.
- FR-7.4 Barber and admin dashboards shall display income totals based on stored income records.

4.8 Feature 8: Feedback and Rating

Customers shall be able to submit service feedback after completed appointments.

- FR-8.1 The system shall allow feedback only for completed appointments.
- FR-8.2 Feedback shall include rating and optional comment.
- FR-8.3 The system shall associate feedback with appointment, customer, and barber.
- FR-8.4 Barbers shall be able to view feedback and reply where supported.

4.9 Feature 9: Barber Posts, Comments, and Reactions

The system shall support barber portfolio posts and customer engagement.

- FR-9.1 Barbers shall create posts with caption and image upload.
- FR-9.2 The system shall store uploaded image URL metadata after Cloudinary upload.

- FR-9.3 Customers and barbers shall view posts in the dashboard feed.
- FR-9.4 Logged-in users shall be able to comment and react to posts according to role rules.
- FR-9.5 Admins shall be able to moderate or monitor post activity where required.

4.10 Feature 10: Admin Monitoring and Management

Admins shall monitor platform performance and manage policy-level operations.

- FR-10.1 Admins shall view overall appointment, user, barber, and revenue statistics.
- FR-10.2 Admins shall approve or decline barber leave requests.
- FR-10.3 Admins shall create, activate, deactivate, or review barber accounts.
- FR-10.4 Admins shall set minimum and maximum haircut price policy.
- FR-10.5 Admins shall monitor owner/platform revenue based on the 20% share.

5. Non-Functional Requirements

Category	Requirement ID	Requirement
Performance	NFR-1	The API shall respond within 2 seconds for common requests under normal demonstration load.
Performance	NFR-2	The customer booking flow shall be completable within 3 minutes by a first-time user with valid inputs.
Security	NFR-3	Passwords and authentication secrets shall not be stored as plain text in the application database.
Security	NFR-4	Protected endpoints shall verify JWT/Bearer tokens and enforce role-based permissions.
Security	NFR-5	Application secrets such as Firebase service account, Cloudinary API secret, and JWT secret shall be stored in environment variables or ignored local files.
Usability	NFR-6	Forms shall show clear validation messages for invalid inputs and failed actions.
Compatibility	NFR-7	The web app shall support current Chrome, Edge, Firefox, and Safari versions; the Android app shall support API 26+ for current implementation.
Reliability	NFR-8	Booking creation and income recording shall use server-side validation to reduce duplicate or inconsistent records.
Maintainability	NFR-9	Backend code shall remain organized by feature/vertical slice to simplify maintenance.
Scalability	NFR-10	Firestore collections and API services shall be structured to support growth in users, appointments, posts, and records.

6. Data Requirements

6.1 Main Firestore Collections / Data Entities

Collection / Entity	Key Data Fields	Purpose
users	userId, firstName, lastName, email, phoneNumber, role, isActive, createdAt, updatedAt	Stores account profile and role mapping.
barber_profiles	barberProfileId, userId, bio, yearsExperience, specialties, gcashNumber, isAvailable, profileImageUrl	Stores barber-specific public and business details.
haircut_styles	haircutStyleId, barberProfileId, name, description, basePrice, durationMinutes, imageUrl, isActive	Stores services offered by barbers.
style_options	styleOptionId, haircutStyleId, name, additionalPrice, additionalTimeMinutes, isActive	Stores add-ons connected to a haircut style.
appointments	appointmentId, customerId, barberProfileId, haircutStyleId, selectedOptionIds, appointmentDateTime, totalPrice, status, paymentMethod	Stores customer booking records.
income_records	incomeRecordId, appointmentId, barberProfileId, grossAmount, barberShare, ownerShare, recordedAt	Stores computed income after completed appointments.
posts	postId, barberProfileId, caption, imageUrl, createdAt, updatedAt	Stores barber social feed posts.
comments	commentId, postId, userId, commentText, createdAt	Stores comments and replies.
reactions	reactionId, postId, userId, type, createdAt	Stores post reaction data.
feedback	feedbackId, appointmentId, customerId, barberProfileId, rating, comment, barberReply, createdAt	Stores customer rating and feedback after service.
leave_requests	leaveRequestId, barberProfileId, dateRange, reason, status, approvedBy, reviewedAt	Stores barber leave requests and admin decisions.
admin_settings	settingsId, minPrice, maxPrice, policyValue, updatedAt	Stores platform-level settings and policies.

6.2 Data Validation Rules

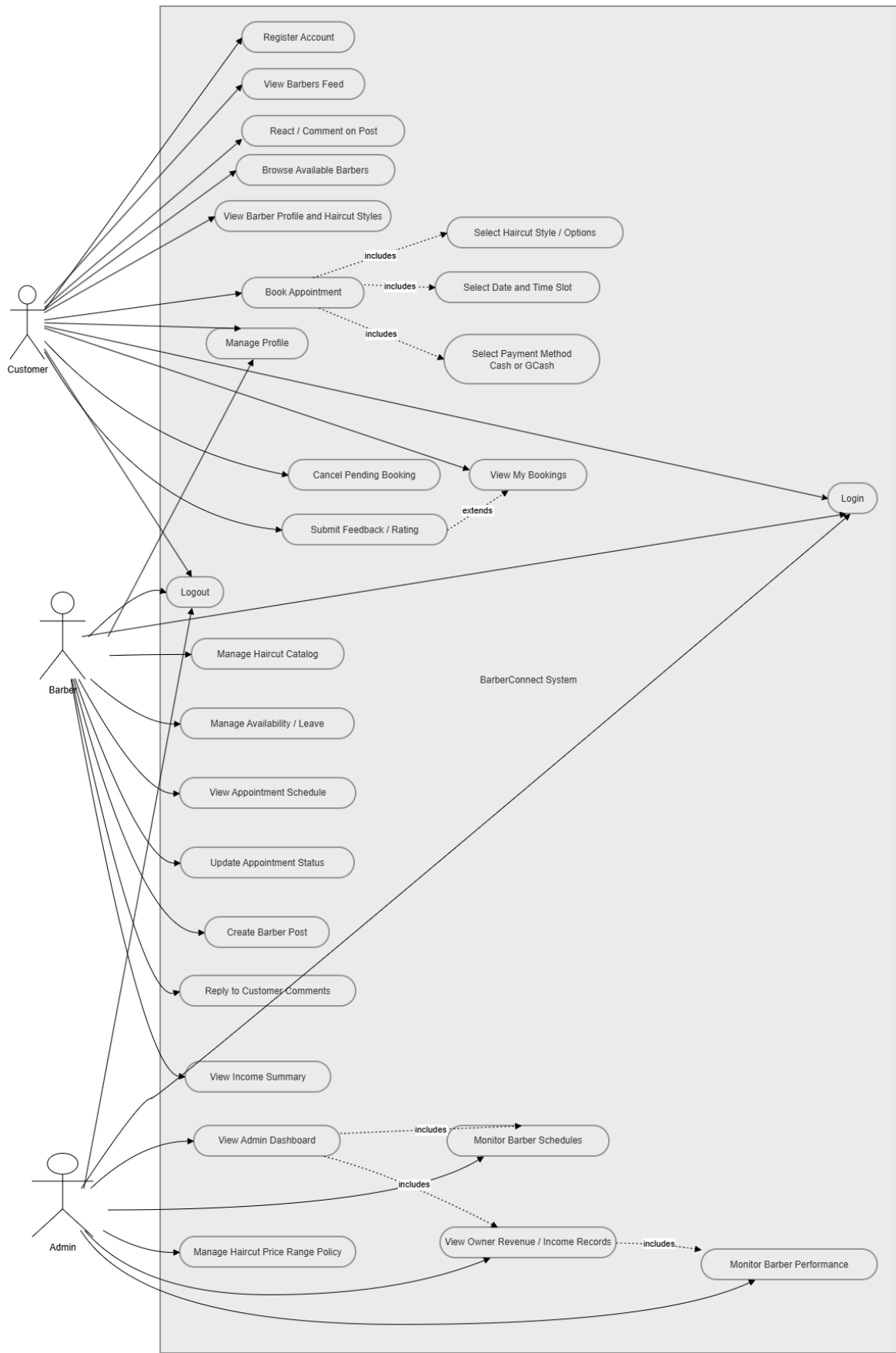
- Email must be unique and valid in format.
- Phone number should follow Philippine mobile number format where possible.
- User role must be CUSTOMER, BARBER, or ADMIN.
- Appointment total price must be non-negative and computed from active catalog records.
- Appointment date/time must not conflict with existing bookings or approved leave.
- Feedback rating must be between 1 and 5.
- Income record must reference a completed appointment.
- Cloudinary media URLs must be stored as URLs, not raw image files in Firestore.

7. Acceptance Criteria

ID	Scenario	Acceptance Criteria
AC-1	Customer Registration	Given a new customer enters valid account details, when registration is submitted, then the system creates an account/profile and redirects the customer to the correct dashboard.
AC-2	Role-Based Login	Given a user logs in successfully, when the role is CUSTOMER, BARBER, or ADMIN, then the system routes the user to the matching dashboard and blocks unauthorized pages.
AC-3	Successful Booking	Given a customer selects an available barber, service, options, date/time, and payment method, when booking is confirmed, then an appointment record is stored and appears in both customer and barber views.
AC-4	Booking Conflict Prevention	Given a selected slot is already booked or the barber is on approved leave, when the customer attempts booking, then the system rejects the request and shows a clear error message.
AC-5	Catalog Management	Given a barber enters valid service details, when the haircut style is saved, then the service appears in the customer-facing catalog.
AC-6	Appointment Completion and Income	Given a barber marks a valid appointment as completed, when the update is accepted, then the system creates one income record with 80% barber share and 20% owner/platform share.
AC-7	Feedback	Given a completed appointment exists, when the customer submits a rating and comment, then feedback is stored and reflected in barber feedback/rating views.
AC-8	Social Feed	Given a barber creates a post with image and caption, when the post is submitted, then it appears in the dashboard feed and supports comments/reactions.
AC-9	Admin Leave Approval	Given a barber submits a leave request, when an admin approves it, then customers cannot book that barber on the approved leave date/range.
AC-10	Admin Revenue Monitoring	Given completed appointments with income records exist, when admin views the dashboard, then total owner/platform income and barber earnings are visible.

8. System Models

8.1 Use Case Diagram



8.2 Customer Booking Activity Diagram

Customer Booking Activity Diagram

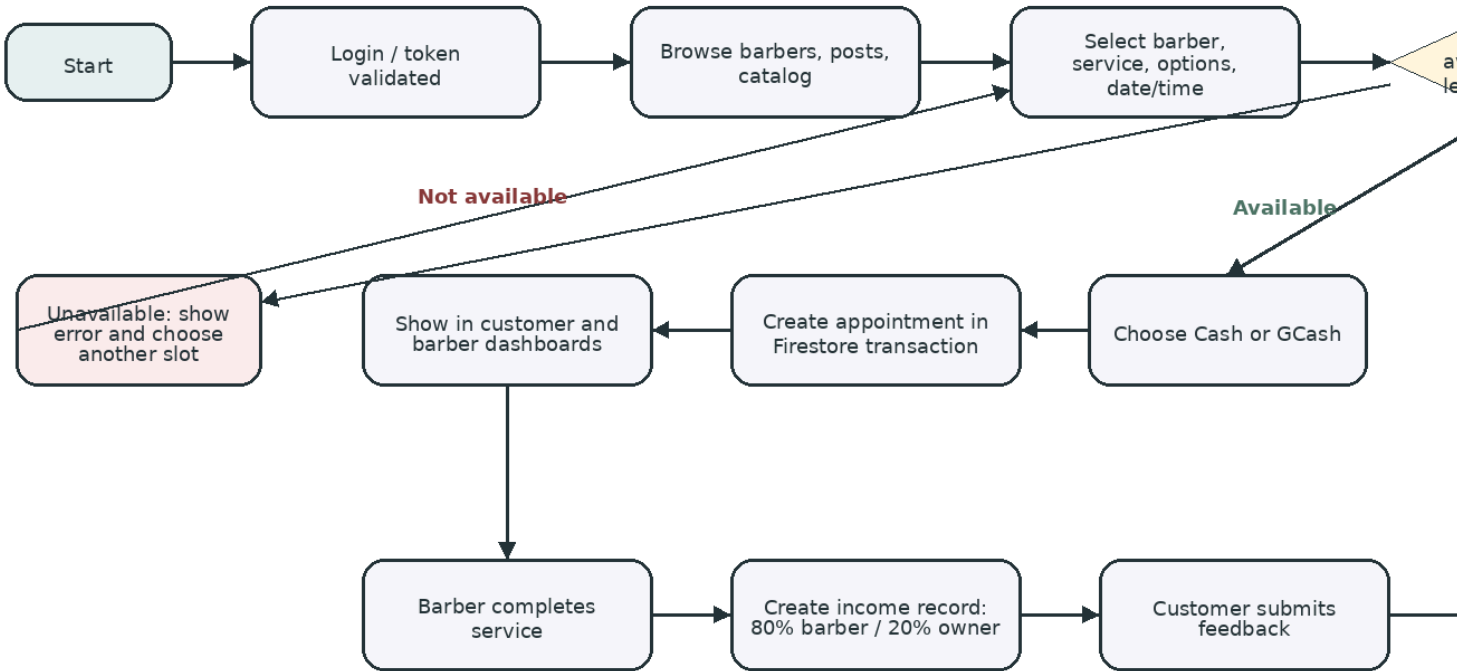


Figure 2. Customer booking and appointment completion activity flow.

8.3 System Architecture Diagram

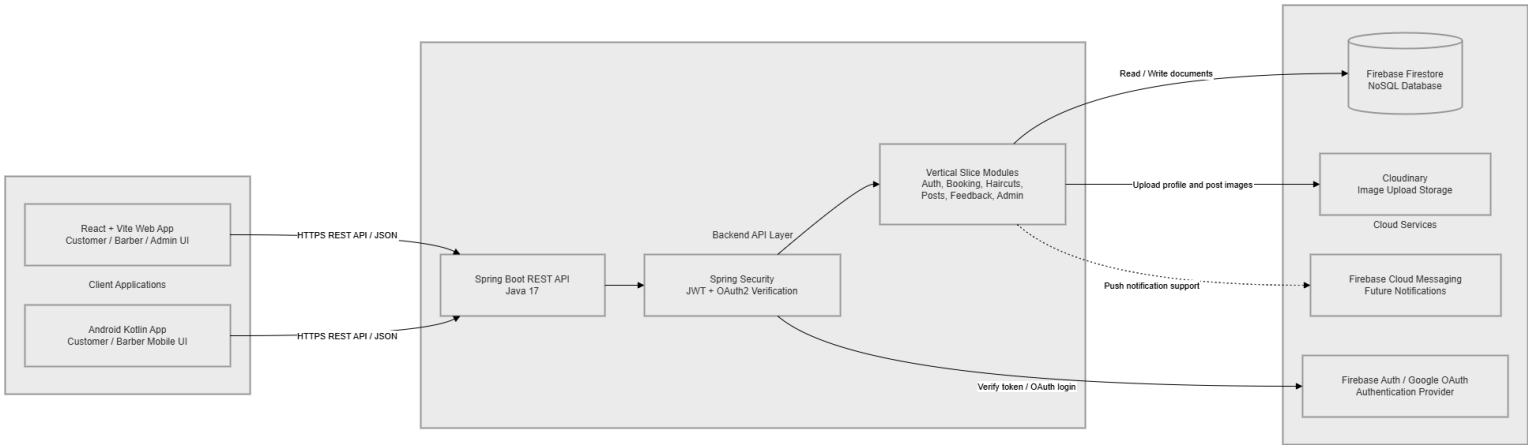


Figure 3. Repository-aligned BarberConnect architecture.

8.4 Firestore NoSQL Data Model / ERD

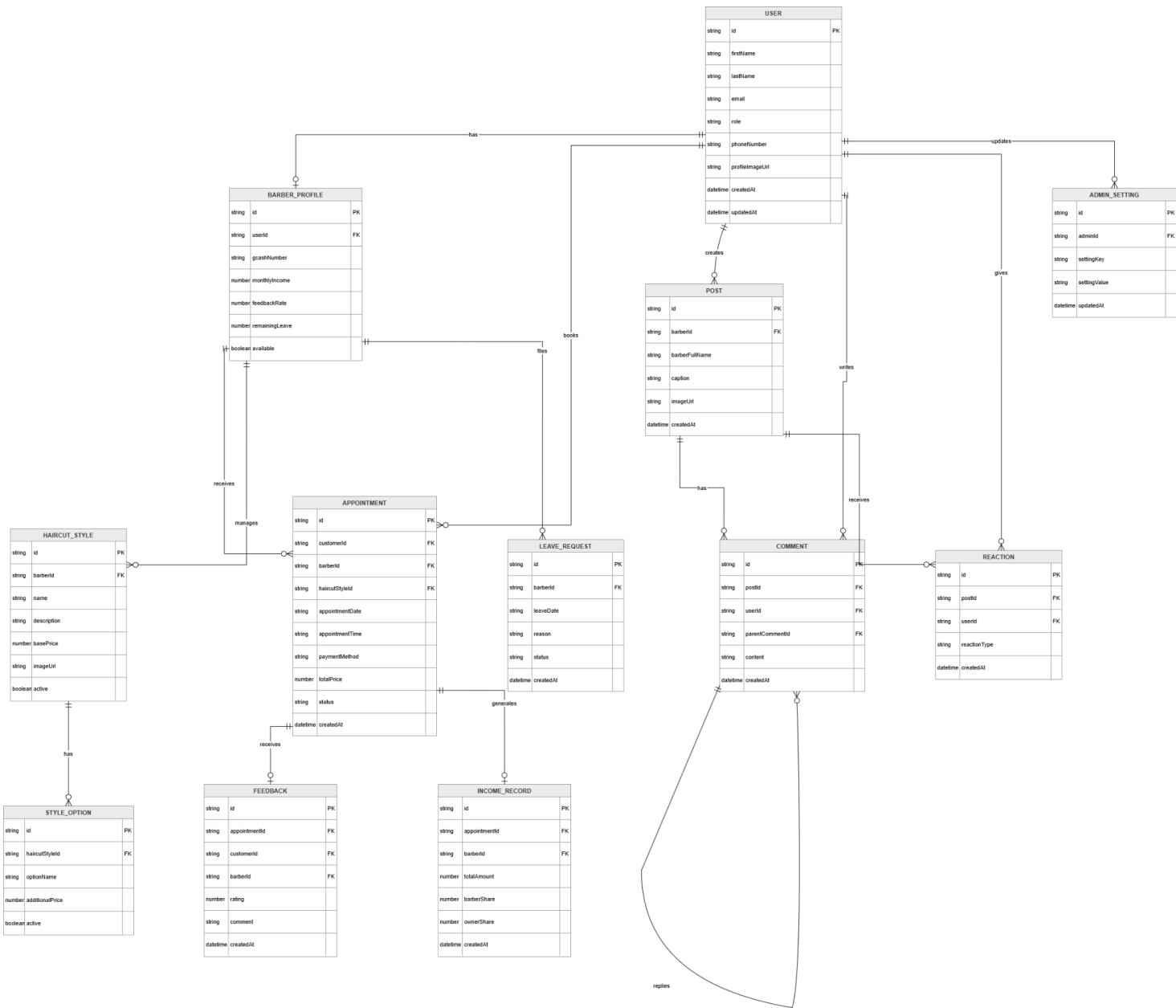


Figure 4. Firestore NoSQL data model and logical relationships.

8.5 Class Diagram

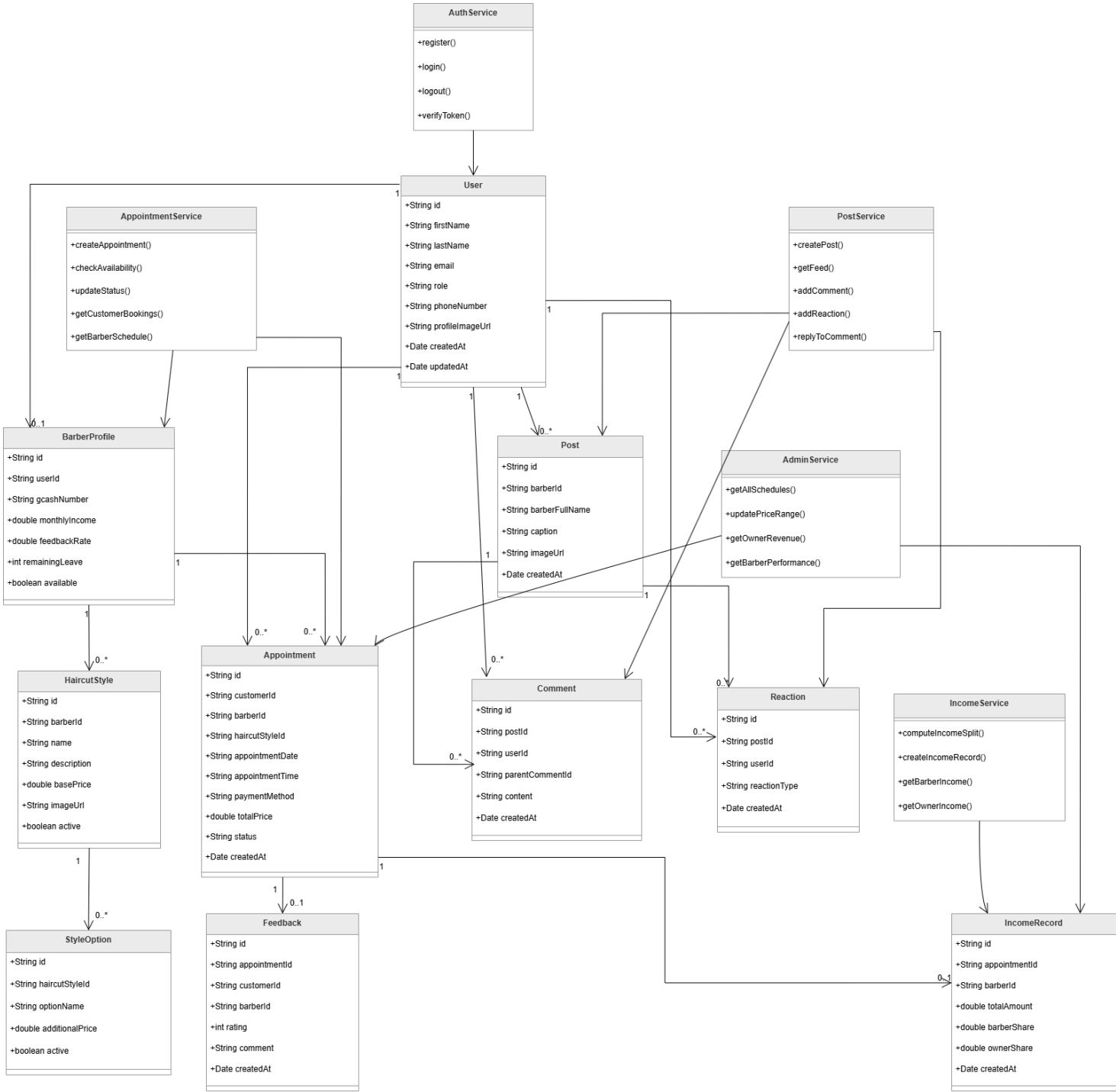


Figure 5. Core BarberConnect domain class model.

8.6 Sequence Diagram: Authentication

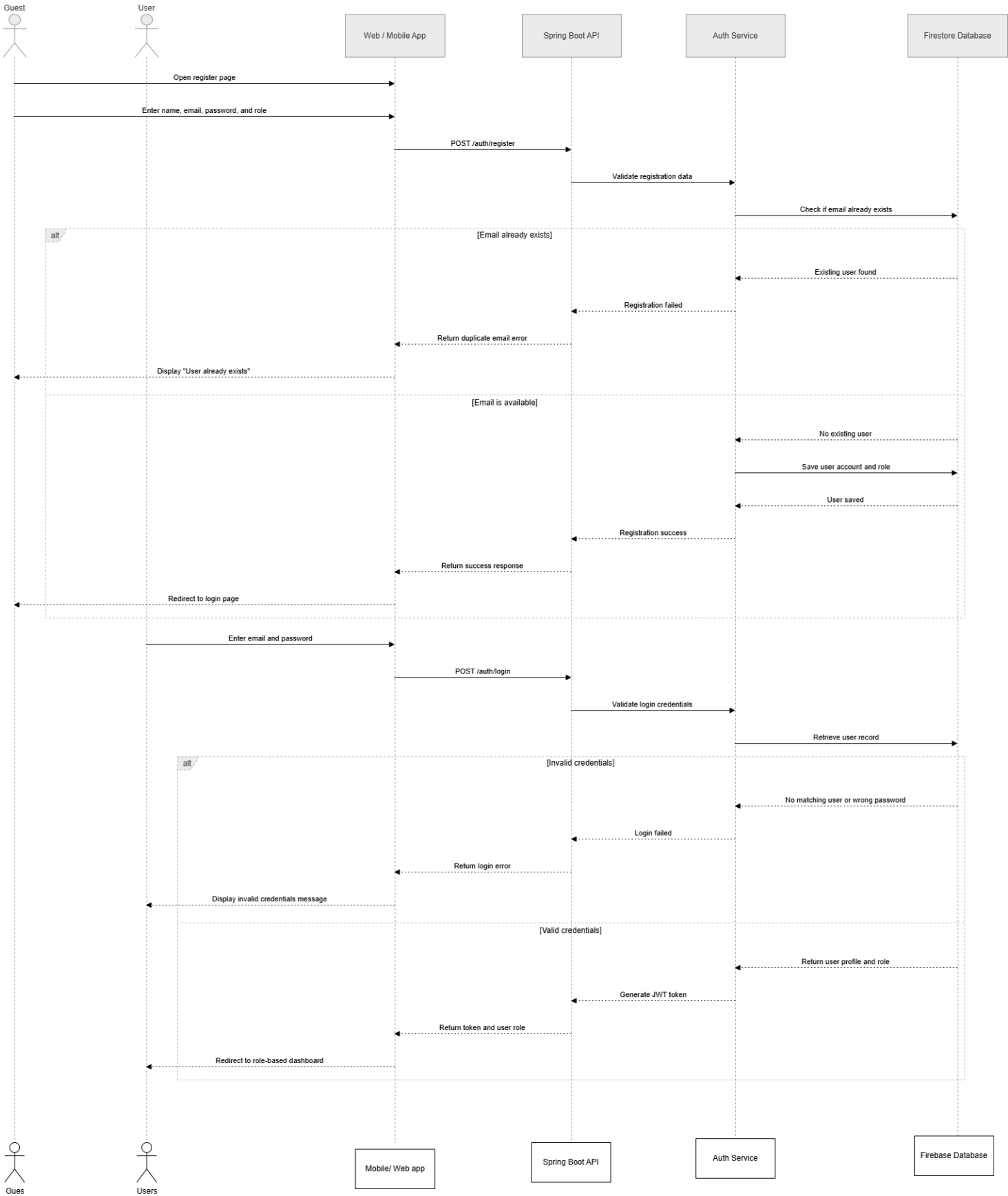


Figure 6. Authentication and role routing sequence.

8.7 Sequence Diagram: Customer Booking

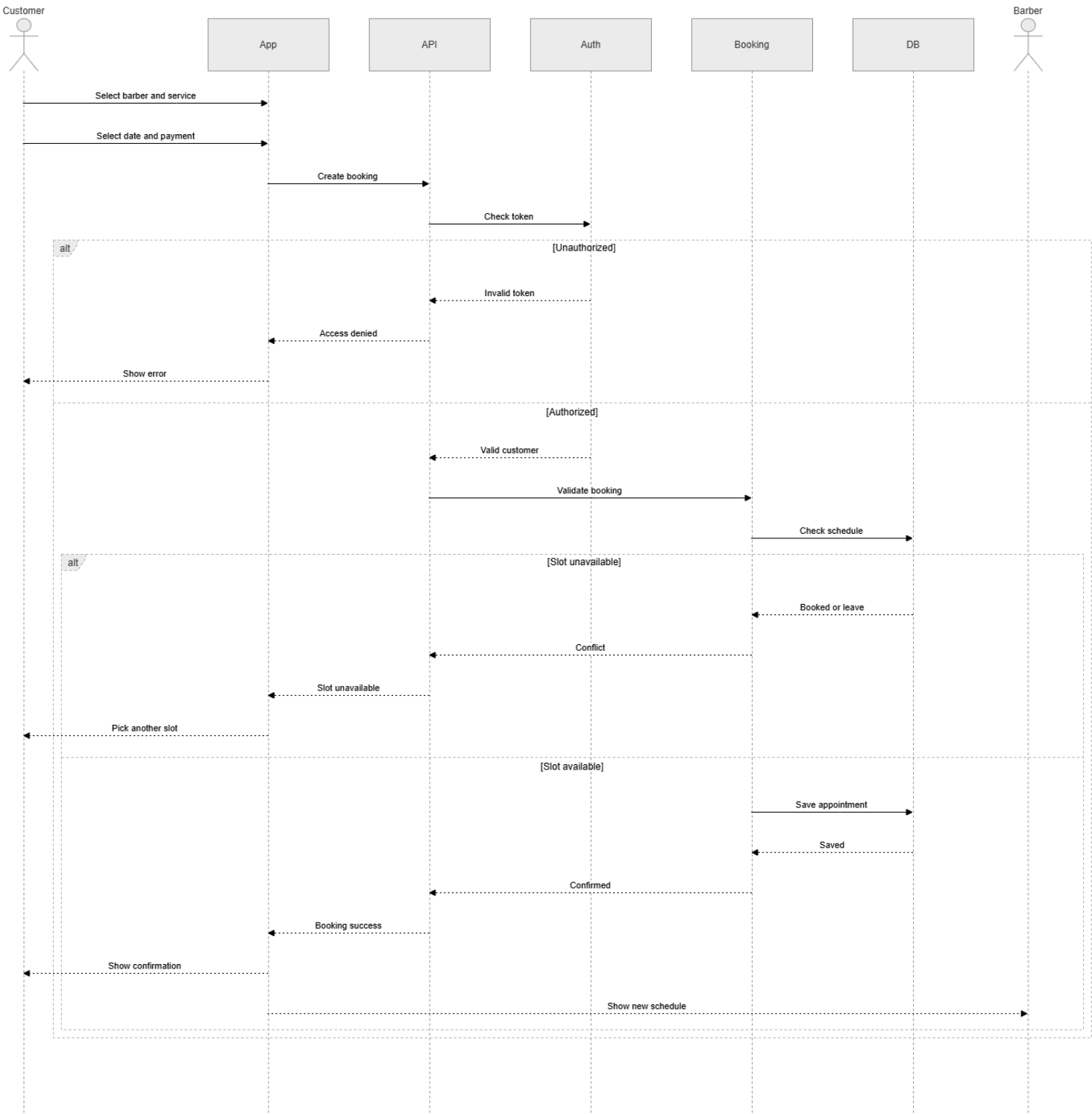


Figure 7. Customer booking sequence.

8.8 Sequence Diagram: Appointment Completion and Income

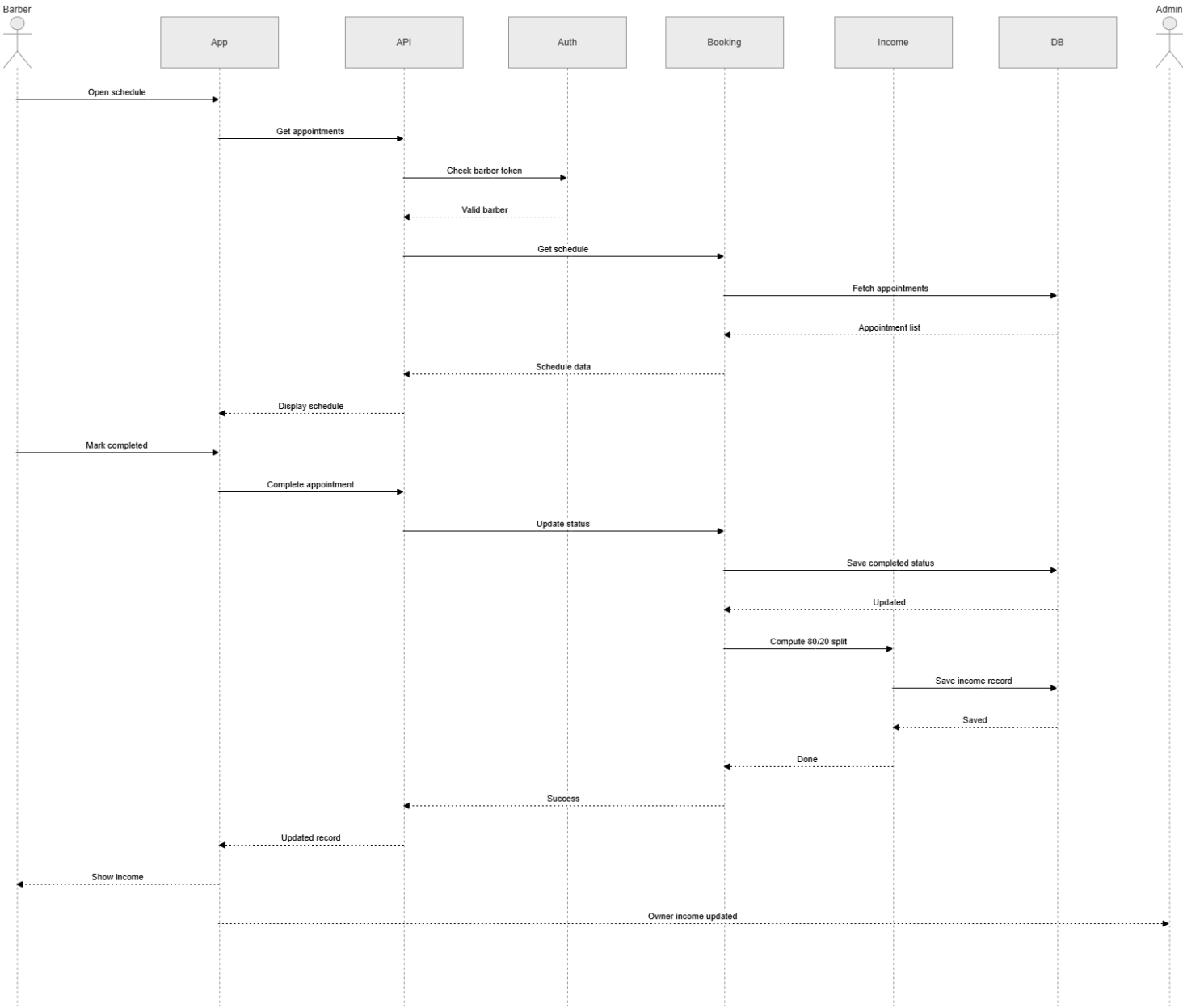
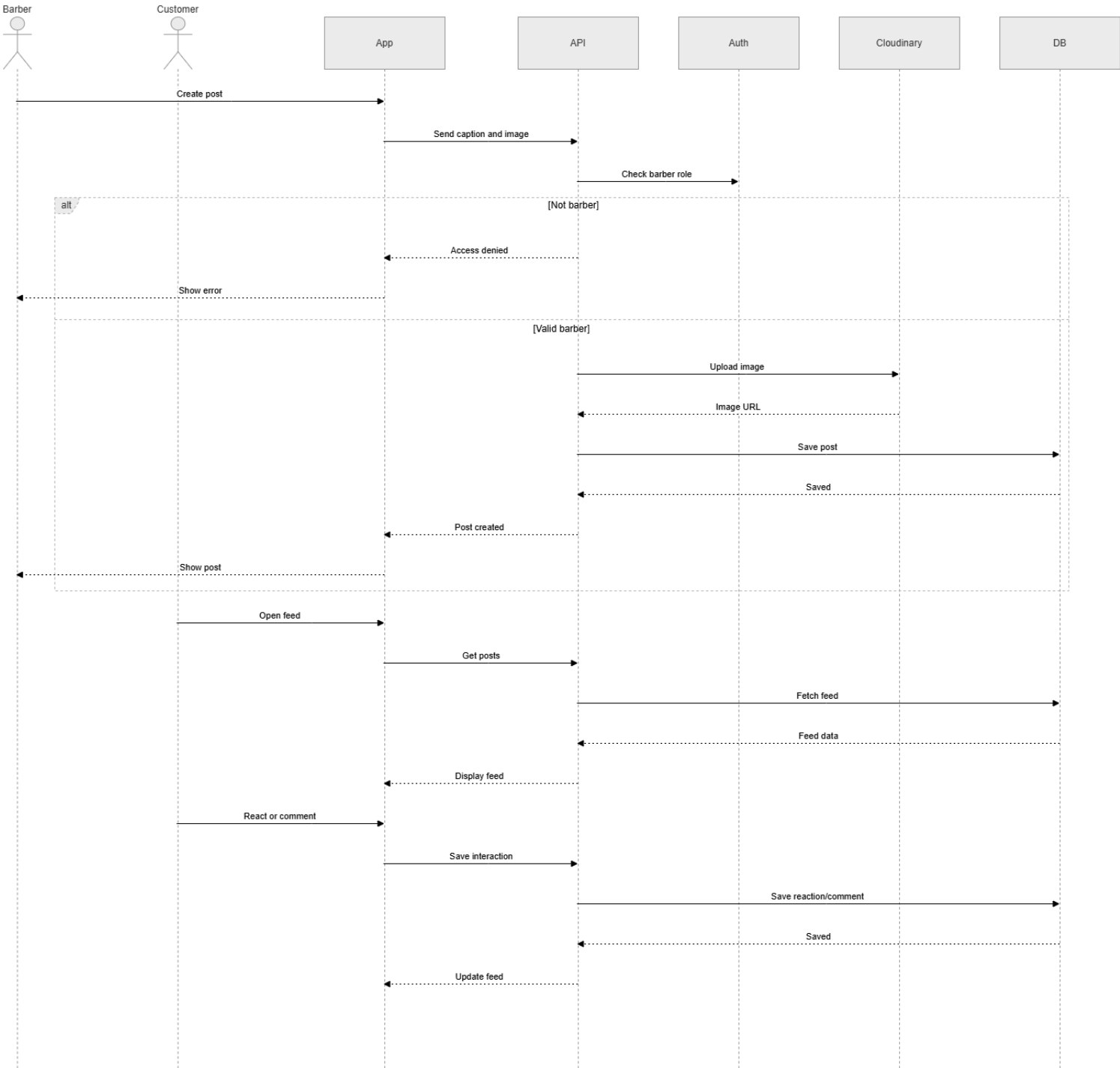
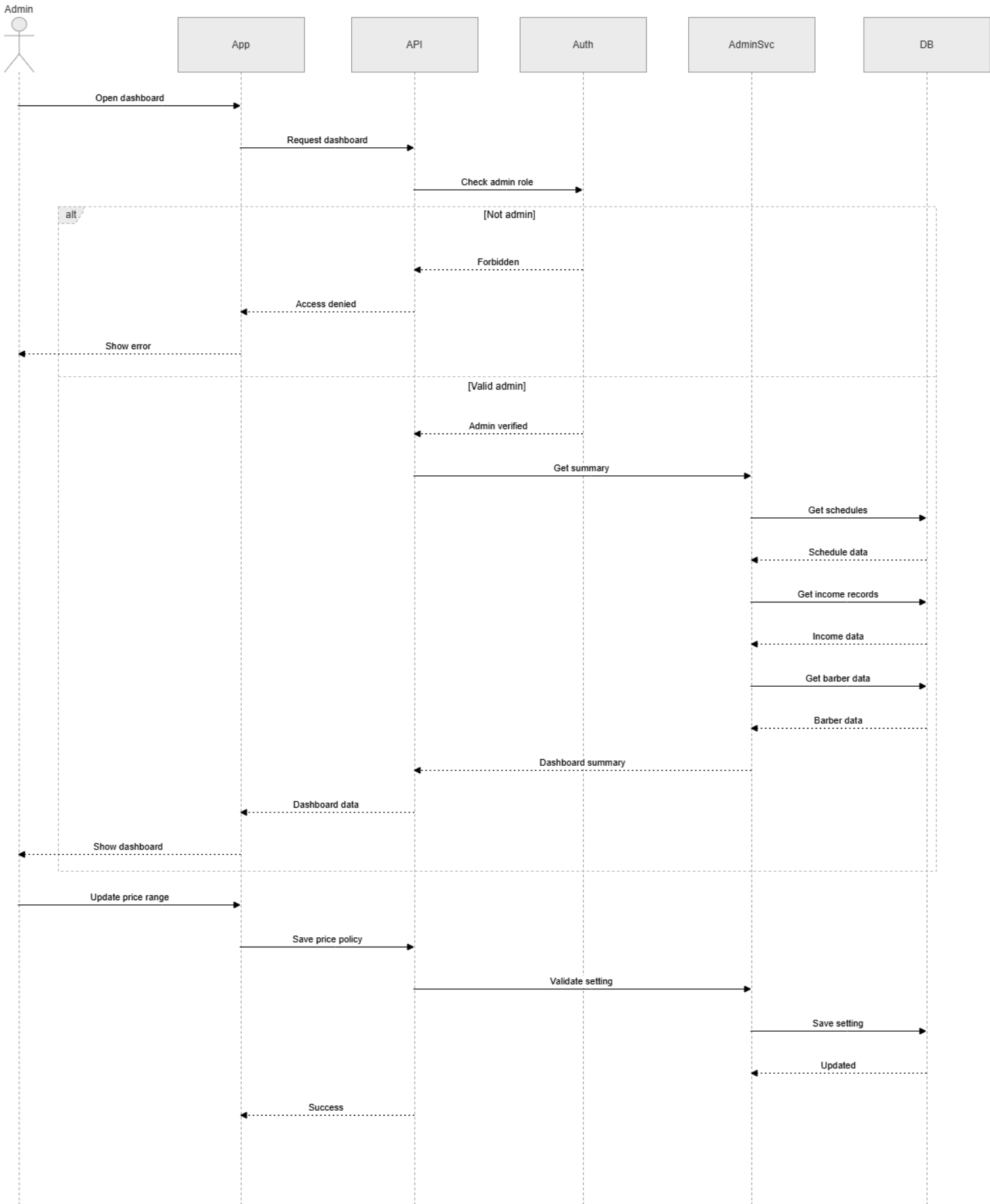


Figure 8. Completion and income record sequence.

8.9 Sequence Diagram: Appointment Completion and Income



8.9 Sequence Diagram: Admin Monitoring Sequence



9. Requirements Traceability Matrix

Module / Requirement Area	Primary Requirements	Related Design / Implementation Area	User Role
Authentication and Accounts	FR-1.1 to FR-1.6	auth vertical slice, Firebase Auth, JWT Utility, login/register UI	Customer, Barber, Admin
RBAC	FR-2.1 to FR-2.4	Spring Security filters/guards, role-based routes	Customer, Barber, Admin
Discovery and Feed	FR-3.1 to FR-3.4, FR-9.1 to FR-9.5	social vertical slice, Cloudinary image upload, dashboard feed	Customer, Barber, Admin
Booking	FR-4.1 to FR-4.6	appointment vertical slice, Firestore appointment collection, booking pages	Customer, Barber
Schedule and Leave	FR-5.1 to FR-5.5, FR-10.2	admin and appointment vertical slices, leave_requests collection	Barber, Admin
Catalog	FR-6.1 to FR-6.4	catalog vertical slice, haircut_styles and style_options collections	Barber, Customer, Admin
Income	FR-7.1 to FR-7.4, FR-10.5	income vertical slice, income_records collection, dashboards	Barber, Admin
Feedback	FR-8.1 to FR-8.4	feedback documents linked to appointments	Customer, Barber
Admin Monitoring	FR-10.1 to FR-10.5	admin dashboard, admin_settings, analytics summaries	Admin

10. Appendices

10.1 Suggested API Endpoint Groups

Endpoint Group	Representative Operations
Authentication	POST /api/v1/auth/register, POST /api/v1/auth/login, GET /api/v1/auth/me, POST /api/v1/auth/google
Barber	GET /api/v1/barbers, GET /api/v1/barbers/{id}, PUT /api/v1/barbers/profile, POST /api/v1/barbers/leave-requests
Catalog	GET /api/v1/catalog, POST /api/v1/catalog/haircuts, PUT /api/v1/catalog/haircuts/{id}, DELETE /api/v1/catalog/haircuts/{id}
Appointments	GET /api/v1/appointments, POST /api/v1/appointments, PUT /api/v1/appointments/{id}/status, DELETE /api/v1/appointments/{id}
Social	GET /api/v1/posts, POST /api/v1/posts, POST /api/v1/posts/{id}/comments, POST /api/v1/posts/{id}/reactions
Feedback	POST /api/v1/feedback, GET /api/v1/barbers/{id}/feedback
Income	GET /api/v1/income/barber/{id}, GET /api/v1/income/admin
Admin	GET /api/v1/admin/statistics, POST /api/v1/admin/barbers, PUT /api/v1/admin/settings, PUT /api/v1/admin/leave-requests/{id}

10.2 Repository Alignment Notes

- The updated SRS now describes the actual BarberConnect platform instead of only a mini authentication app.
- The technology stack aligns with the current project README: Spring Boot backend, Firestore, Cloudinary, React/Vite web, Android Kotlin mobile, JWT/OAuth2, and vertical slice architecture.
- The data model aligns with the repository ERD file and the updated SDD diagram: users, barber_profiles, haircut_styles, style_options, appointments, income_records, posts, comments, reactions, feedback, leave_requests, and admin_settings.
- The requirements support the project demonstration flow: login, customer booking, barber management, completion, income record, social feed, feedback, and admin monitoring.